

SDN-based Intrusion Detection & Mitigation System

ARP Spoofing e Port Scanning Detection in ambiente SDN (Mininet + Ryu, OpenFlow 1.3)

Studente: Francesco Sabatino

Matricola: DE9000191

Repository GitHub: <https://github.com/Fraykk/SDN-based-Intrusion-Detection-Mitigation-System>

Corso: Network and Cloud Infrastructures — A.A. 2025/2026

1 Obiettivo del progetto

Il progetto realizza, in un ambiente SDN emulato con Mininet e controllato da Ryu (OpenFlow 1.3), un sistema per il rilevamento e la mitigazione di due classi di attacco: *ARP spoofing*, cioè una violazione del binding IP-MAC tipica degli attacchi di tipo MITM, e *port scanning*, in cui un host sonda molte porte o destinazioni in un intervallo di tempo ridotto.

Invece di applicare una soglia statica all'intero traffico, il sistema mette in quarantena il solo host responsabile della violazione e lo sblocca automaticamente allo scadere del periodo configurato, se non si verificano ulteriori violazioni, limitando così l'impatto sui nodi legittimi e il rischio di *over-blocking*.

2 Topologia di rete

La topologia utilizzata nel testbed è composta da tre switch, un controller Ryu e cinque host (Figure 1). I link di accesso tra host e switch sono configurati a 10 Mbps, mentre i collegamenti della dorsale sono a 100 Mbps.

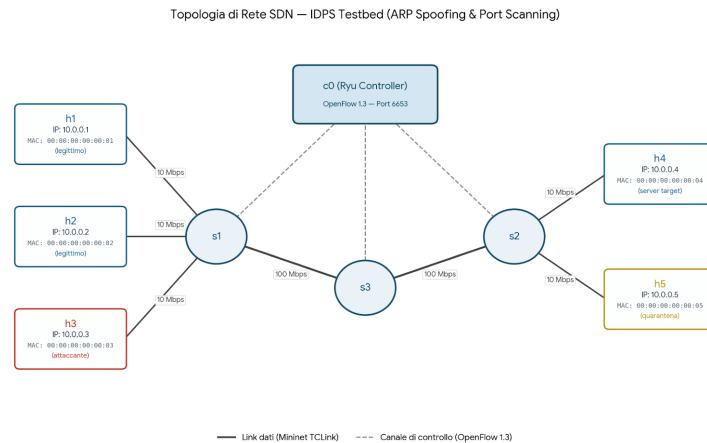


Figura 1: Topologia di rete SDN utilizzata per i test.

Tabella 1: Ruolo dei nodi della topologia.

Nodo	IP	Ruolo
h1	10.0.0.1	Host legittimo
h2	10.0.0.2	Host legittimo
h3	10.0.0.3	Attaccante
h4	10.0.0.4	Server / bersaglio
h5	10.0.0.5	Host di quarantena
s1 / s2 / s3	—	Switch di accesso / rete server / dorsale
c0	—	Controller Ryu remoto (OpenFlow 1.3)

3 Design del sistema

Il controller è organizzato in moduli separati, per distinguere monitoraggio, decisione e applicazione delle contromisure (Table 2).

Tabella 2: Moduli principali del controller.

Modulo	Responsabilità
ArpSpoofDetector	Gestisce la tabella dei binding IP-MAC “trusted” e genera un allarme quando un IP già noto viene associato a un MAC differente.
PortScanDetector	Mantiene una <i>sliding window</i> per host sorgente sulle coppie distinte (<i>dst_ip</i> , <i>dst_port</i>).
MitigationManager	Gestisce la tabella di quarantena condivisa, installa/rimuove le regole OpenFlow e provvede allo sblocco allo scadere del timeout.
SecurityIDPSApp	Applicazione Ryu principale: implementa il learning switch L2 e integra i moduli di rilevamento nella gestione dei <i>packet_in</i> .

3.1 Parametri di configurazione

Tabella 3: Parametri di configurazione e motivazione della scelta.

Parametro	Valore	Motivazione
PORT_SCAN_WINDOW_SEC	5 s	Finestra breve a sufficienza da non accumulare traffico interattivo distribuito nel tempo, ma ampia abbastanza da rilevare tool automatici come Nmap.
PORT_SCAN_THRESHOLD	8	Soglia superiore alle connessioni concorrenti tipiche di un host in attività lecite, ma molto inferiore alle sonde generate anche da una scansione ridotta (es. 100 porte).
QUARANTINE_TIMEOUT_SEC	30 s	Periodo che consente lo sblocco automatico, limitando il rischio di blocco permanente in caso di falso positivo.

4 Test e risultati

4.1 Baseline (nessun attacco)

Prima di iniettare traffico malevolo è stata verificata la connettività della topologia in condizioni ordinarie. Il comando `pingall` ha restituito 0% di pacchetti persi (20/20 ricevuti), indicando che il controller popola correttamente le tabelle dei flussi tramite il learning L2. È stato quindi eseguito un test prestazionale TCP tra host legittimi (server `iperf` su `h1`, 10.0.0.1:5001, client su `h2`), con throughput medio di 7.64 Mbit/s, coerente con il limite di 10 Mbps dei link di accesso. Durante lo scambio non è stato generato alcun allarme dal modulo IDPS.

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5
h2 -> h1 h3 h4 h5
h3 -> h1 h2 h4 h5
h4 -> h1 h2 h3 h5
h5 -> h1 h2 h3 h4
*** Results: 0% dropped (20/20 received)
mininet> h1 iperf -s &
mininet> h2 iperf -c h1
-----
Client connecting to 10.0.0.1, TCP port 5001
TCP window size: 128 KByte (default)
-----
[ 1] local 10.0.0.2 port 37070 connected with 10.0.0.1 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-12.2096 sec 11.1 MBytes 7.64 Mbits/sec
```

Figura 2: Output del test di baseline: `pingall` e throughput `iperf`.

Procedura. Tramite un terminale dedicato (`xterm h3`) è stato eseguito un attacco di ARP spoofing verso h4 (10.0.0.4), falsificando l'identità di h1 (10.0.0.1):

Risultati. Il modulo `ArpSpoofDetector` ha confrontato l’annuncio con la tabella dei binding appresa in baseline: 10.0.0.1 risultava associato al MAC 00:00:00:00:00:01, mentre l’annuncio riportava il MAC malevolo 00:00:00:00:00:03, generando l’allarme. Il `MitigationManager` ha istruito lo switch s1 a inserire una regola OpenFlow ad alta priorità (`priority=100`, `match su eth_src`) che scarta il traffico dell’attaccante. La verifica con `h3 ping -c 2 h4` ha prodotto il 100% di pacchetti persi, confermando l’isolamento di h3, senza degradazioni per le comunicazioni tra host legittimi.

Figura 3: Output di arpspoof

Figura 4: Log del rilevamento ARP spoofing e inserimento in quarantena

Procedura. Dall'host malevolo h3 è stata avviata una scansione delle porte TCP verso il server h4 (10.0.0.4):

Risultati. Il modulo `PortScanDetector` ha analizzato i `packet_in` TCP popolandolo una *sliding window* di 5 secondi (`PORT_SCAN_WINDOW_SEC`). Raggiunta la soglia di 8 combinazioni distinte (`dst_ip`, `dst_port`), il sistema ha generato l'allarme e il `MitigationManager` ha installato su `s1` una regola restrittiva (priorità 100), interrompendo la scansione; Nmap ha rilevato 100 porte filtrate.

```

root@NCIS:/home/vboxuser/Desktop/sdn_project# nmap -p 1-100 10.0.0.4
Starting Nmap 7.80 ( https://nmap.org ) at 2026-09-04 18:09 CEST
Nmap scan report for 10.0.0.4
Host is up (0.021s latency).
All 100 scanned ports on 10.0.0.4 are filtered
MAC Address: 00:00:00:00:00:04 (Xerox)

Nmap done: 1 IP address (1 host up) scanned in 16.32 seconds

```

Figura 5: Output della scansione Nmap

```

[MITIGATION] Host 00:00:00:00:00:03 messo in QUARANTENA (Port scan sospetto: 10.
0.0.3 ha contattato 8 combinazioni (ip,porta) distinte in 5s)
[MITIGATION] Timeout scaduto: sblocco host 00:00:00:00:00:03

```

Figura 6: Log del PortScanDetector con relativa quarantena

4.4 Falsi positivi

Procedura. È stato generato traffico ordinario, a ritmo umano, dagli host leciti h1 e h2 verso h4 (10.0.0.4) e altri nodi, alternando richieste ICMP (h1 ping -c 3 h4) e connessioni TCP ordinarie.

Risultati. Il ping ha riportato 0% di packet loss con RTT medio stabile (circa 7 ms). Il controller non ha registrato alcun allarme né attivato quarantene: il rate di nuove sessioni TCP è rimasto sotto la soglia configurata (PORT_SCAN_THRESHOLD = 8 coppie distinte in 5 s). Non sono stati osservati falsi positivi durante la normale operatività.

```

mininet> h2 nc -zv 10.0.0.4 5001
nc: connect to 10.0.0.4 port 5001 (tcp) failed: Connection refused
mininet> h2 iperf -c 10.0.0.1 -t 1
-----
Client connecting to 10.0.0.1, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[  1] local 10.0.0.2 port 35592 connected with 10.0.0.1 port 5001
[ ID] Interval           Transfer     Bandwidth
[  1] 0.0000-1.4413 sec   1.63 MBytes  9.46 Mbits/sec

```

Figura 7: Traffico TCP durante la verifica dell'assenza di falsi positivi

4.5 Sblocco dinamico

Procedura. Terminata l'attività malevola di h3, il comportamento del controller è stato osservato per un intervallo superiore a QUARANTINE_TIMEOUT_SEC (30 s), seguito da un nuovo test di connettività (h3 ping -c 3 10.0.0.4).

Risultati. Il thread periodico del MitigationManager (_monitor, sweep ogni 5 s) ha rilevato la scadenza della quarantena per il MAC 00:00:00:00:00:03, revocando tramite un messaggio OpenFlow OFPFlowMod (comando OFPFC_DELETE) la regola di drop su s1. Il ping successivo ha registrato 0% di pacchetti persi (3/3, RTT medio $\approx$ 3.1 ms), confermando il ripristino automatico del servizio senza intervento manuale.

```
mininet> h3 ping -c 3 10.0.0.4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
64 bytes from 10.0.0.4: icmp_seq=1 ttl=64 time=8.25 ms
64 bytes from 10.0.0.4: icmp_seq=2 ttl=64 time=1.12 ms
64 bytes from 10.0.0.4: icmp_seq=3 ttl=64 time=0.084 ms

--- 10.0.0.4 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 0.084/3.154/8.254/3.631 ms
```

Figura 8: Ripristino della connettività dopo la scadenza della quarantena (destra).

5 Conclusioni ed estensioni possibili

Il lavoro svolto ha mostrato, nei test eseguiti, il funzionamento di un'architettura IDPS programmabile in ambiente SDN tramite OpenFlow 1.3. La separazione tra `ArpSpoofDetector`, `PortScanDetector` e `MitigationManager` consente di distinguere il rilevamento L2/L4 dalla fase di enforcement e di applicare la quarantena al singolo MAC responsabile, senza coinvolgere gli host legittimi. La gestione asincrona della quarantena tramite thread dedicato permette inoltre una politica reversibile: allo scadere del timeout la regola di drop viene rimossa e il forwarding ripristinato automaticamente.

Come possibili estensioni si individuano:

- **Remediation attiva e captive portal su h5** (10.0.0.5), riscrivendo i campi di intestazione dei pacchetti (`set_field`) per redirigere l'host sanzionato verso una pagina informativa o un honeypot, invece del semplice drop;
- **Whitelisting dinamico** per nodi infrastrutturali critici (gateway, server DNS/web), a riduzione del rischio di blocchi accidentali;
- **Persistenza e dashboard di telemetria**, esportando gli eventi di violazione su file strutturati (JSON/CSV) o interfaccia web con metriche di banda e anomalie.